

Глава 14

Регулярные выражения

В главе описывается библиотека регулярных выражений, позволяющая использовать метасимволы (wildcards) и шаблоны (patterns) для поиска и замены символов в строках.

С помощью регулярных выражений можно выполнять ряд операций.

- **Сравнивать** входную строку с регулярным выражением.
- **Искать** шаблоны, соответствующие регулярному выражению.
- **Размечать** символы в соответствии с разделителем токенов, заданным в виде регулярного выражения.
- **Заменять** первую или все подпоследовательности, соответствующие регулярному выражению.

Для всех этих операций можно использовать разные грамматики, которые используются для определения регулярного выражения.

Глава начинается с краткого перечисления разных операций, затем обсуждаются разные грамматики, а в заключение приводятся подробные описания операций над регулярными выражениями.

14.1. Интерфейс сравнения и поиска регулярных выражений

Рассмотрим, как можно проверить, соответствует ли последовательность символов полностью или частично конкретному регулярному выражению:

```
// regex/regex1.cpp

#include <regex>
#include <iostream>
using namespace std;

void out (bool b)
{
    cout << ( b ? "found" : "not found") << endl;
}

int main()
{
    // ищем значение с дескрипторами XML/HTML (используя синтаксис по умолчанию):
    regex reg1("<.*>.*/.*>");
    bool found = regex_match ("<tag>value</tag>", // данные
                              reg1);              // регулярное выражение
```

```

out(found);

// ищем значение с дескрипторами XML/HTML
// (дескрипторы до и после значения должны совпадать):
regex reg2("<(.*>.*</\\1>");
found = regex_match("<tag>value</tag>", // данные
                    reg2); // регулярное выражение
out(found);

// ищем значение с дескрипторами XML/HTML (используя синтаксис команды grep):
regex reg3("<\\(.*\\)>.*</\\1>", regex_constants::grep);
found = regex_match("<tag>value</tag>", // данные
                    reg3); // регулярное выражение
out(found);

// используем C-строку как регулярное выражение
// (требуется явное приведение к regex):
found = regex_match("<tag>value</tag>", // данные
                    regex("<(.*>.*</\\1>)")); // регулярное выражение
out(found);
cout << endl;

// сравнение regex_match() и regex_search():
found = regex_match("XML tag: <tag>value</tag>",
                    regex("<(.*>.*</\\1>)")); // нет соответствия
out(found);

found = regex_match("XML tag: <tag>value</tag>",
                    regex(".*<(.*>.*</\\1>.*")); // есть соответствие
out(found);

found = regex_search("XML tag: <tag>value</tag>",
                    regex("<(.*>.*</\\1>")); // есть соответствие
out(found);

found = regex_search("XML tag: <tag>value</tag>",
                    regex(".*<(.*>.*</\\1>.*")); // есть соответствие
out(found);
}

```

Сначала мы подставляем необходимый заголовочный файл и глобальные идентификаторы пространства имен `std`:

```

#include <regex>
using namespace std;

```

Затем показано, как определить регулярное выражение для проверки соответствия символьной последовательности конкретному шаблону. Мы объявляем и инициализируем объект `reg1` как регулярное выражение:

```

regex reg1("<.*>.*</.*>");

```

Объект, представляющий регулярное выражение, имеет тип `std::regex`. Как и при работе со строками, этот тип представляет собой специализацию класса

`std::basic_regex<>` для символьного типа `char`. Для символьного типа `wchar_t` предусмотрен класс `std::wregex`.

Объект `regl` инициализируется регулярным выражением

```
<.*>.*</.*>
```

Эти регулярные выражения проверяют строку на соответствие шаблону “<символы>символы</символы>”, используя синтаксис `.*`, где символ “.” означает “любой символ за исключением символа новой строки”, а символ “*” означает “ни разу или несколько раз”. Таким образом, мы пытаемся проверять соответствие формату XML- или HTML-дескрипторов. Последовательность символов `<tag>value</tag>` соответствует этому шаблону, поэтому выражение

```
regex_match ("<tag>value</tag>", // данные
            regl);                // регулярное выражение
```

равно `true`.

Можно даже указать, что начальные и заключительные дескрипторы должны быть одинаковыми символьными последовательностями, что демонстрируют следующие операторы:

```
regex reg2("<(.*>.*</\\1>");
found = regex_match ("<tag>value</tag>", // данные
                    reg2);                // регулярное выражение
```

Выражение `regex_match()` снова равно `true`.

Здесь используется концепция группировки. Мы используем конструкцию “(...)” для определения так называемой *группы захвата* (capture group), на которую впоследствии ссылаемся с помощью регулярного выражения “\\1”. Однако мы задаем регулярное выражение как обычную символьную последовательность, поэтому должны определить “символ \, за которым следует символ 1”, как “\\1”. В качестве альтернативы можно было бы использовать *неформатированную строку* (raw string), введенную в стандарте C++11 (см. раздел 3.1.6):

```
R"(<(.*>.*</\\1>)" // эквивалент: "<(.*>.*</\\1>"
```

Такая неформатированная строка позволяет определить символьную последовательность, написав ее точное содержание как неформатированную последовательность символов. Она начинается с символов “R” (“ и заканчивается символами “) ”. Для того чтобы иметь возможность использовать символы “) ” в неформатированной строке, можно использовать разделитель. Таким образом, полный синтаксис неформатированной строки имеет вид `R"delim (...) delim"`, где *delim* — последовательность символов, состоящих не более чем из 16 основных символов, за исключением обратной косой черты, пробелов и скобок.

Здесь мы вводим специальные символы для регулярных выражений как часть грамматики. Стандартная библиотека C++ поддерживает разные грамматики. По умолчанию используется грамматика “модифицированная грамматика ECMAScript,” подробно описанная в разделе 14.8. Следующие строки демонстрируют использование другой грамматики:

```
regex reg3("<\\(.*\\)>.*</\\1>", regex_constants::grep);
found = regex_match ("<tag>value</tag>", // данные
                    reg3);                // регулярное выражение
```

Здесь необязательный второй аргумент `regex_constants::grep` конструктора `regex` задает грамматику, напоминающую команду UNIX `grep`, где, например, необходимо задавать маску группирования символов с помощью дополнительных обратных косых черт (которые в обычных строковых литералах необходимо дополнять обратными косыми чертами). Различия между разными грамматиками рассматривается в разделе 14.9.

Для регулярного выражения все предыдущие примеры использовали отдельный объект. Это не обязательно, однако простой передачи строки или строкового литерала в виде регулярного выражения недостаточно. Хотя в программе объявлено неявное преобразование типа, итоговое выражение не компилируется, потому что оно неоднозначно. Рассмотрим пример:

```
regex_match("<tag>value</tag>", // ОШИБКА: неоднозначность
            "<(.*>.*</\\1>")
regex_match(string("<tag>value</tag>"), // ОШИБКА: неоднозначность
            "<(.*>.*</\\1>")
regex_match("<tag>value</tag>", // ОК
            regex("<(.*>.*</\\1>"))
```

В заключение рассмотрим различия между функциями `regex_match()` и `regex_search()`:

- функция `regex_match()` проверяет, соответствует ли символьная последовательность регулярному выражением *полностью*;
- функция `regex_search()` проверяет, соответствует ли символьная последовательность регулярному выражению *частично*.

Других различий нет. Таким образом, оператор

```
regex_search(data, regex(pattern))
```

всегда эквивалентен оператору

```
regex_match(data, regex("(.|\\n) *"+pattern+"(.|\\n) *"))
```

где символы `"(.|\\n) *"` означают любое количество любых символов (символ `"."` означает любой символ, за исключением символа перехода на новую строку, а символ `"|"` означает "или").

Читатели могли бы возразить, что эти выражения не дают важную информацию, по крайней мере в случае функции `regex_search()`: где именно символьная последовательность соответствует регулярному выражению. Для реализации этой и многих других возможностей были введены новые версии функций `regex_match()` и `regex_search()`, в которых новый параметр возвращает всю необходимую информацию о соответствии.

14.2. Работа с подвыражениями

Рассмотрим следующий пример:

```
// regex/regex2.cpp

#include <string>
#include <regex>
#include <iostream>
#include <iomanip>
```



```

using namespace std;

int main()
{
    string data = "XML tag: <tag-name>the value</tag-name>.";
    cout << "data: " << data << "\n\n";

    smatch m; // для возвращаемой информации о соответствии
    bool found = regex_search (data,
                               m,
                               regex("<(.*?)>(.*?)</(\\1)>"));

    // выводим детали соответствий
    cout << "m.empty():      " << boolalpha << m.empty() << endl;
    cout << "m.size():      " << m.size() << endl;
    if (found) {
        cout << "m.str():      " << m.str() << endl;
        cout << "m.length():   " << m.length() << endl;
        cout << "m.position():  " << m.position() << endl;
        cout << "m.prefix().str(): " << m.prefix().str() << endl;
        cout << "m.suffix().str(): " << m.suffix().str() << endl;
        cout << endl;

        // перебор всех соответствий (с помощью индекса соответствий)
        for (int i=0; i<m.size(); ++i) {
            cout << "m[" << i << "].str():  " << m[i].str() << endl;
            cout << "m.str(" << i << "):      " << m.str(i) << endl;
            cout << "m.position(" << i << "): " << m.position(i)
                << endl;
        }
        cout << endl;

        // перебор всех соответствий (с помощью итератора)
        cout << "matches:" << endl;
        for (auto pos = m.begin(); pos != m.end(); ++pos) {
            cout << " " << *pos << " ";
            cout << "(length: " << pos->length() << ")" << endl;
        }
    }
}

```

В этом примере демонстрируется использование объектов `match_results`, которые можно передать функциям `regex_match()` и `regex_search()` для получения деталей соответствий. Класс `std::match_results<>` — это шаблон, который должен конкретизироваться типом итератора для обрабатываемых символов. Стандартная библиотека C++ содержит несколько заранее определенных конкретизаций:

- `smatch`: для деталей соответствий в строках;
- `cmatch`: для деталей соответствий в C-строках (`const char*`);
- `wsmatch`: для деталей соответствий в объектах класса `wstrings`;
- `wcmatch`: для деталей соответствий в широких C-строках (`const wchar_t*`).

Таким образом, если применить функцию `regex_match()` или `regex_search()` к строкам C++, то следует использовать тип `smatch`, а для обычных строковых литералов должен использоваться тип `cmatch`.

В примере показано, что выводит объект `match_results`, который выполняет поиск регулярного выражения

```
<(.*)>(.*)</(\1)>
```

в строке `data`, инициализированной следующей символьной строкой:

```
"XMLtag: <tag-name>the value</tag-name>
```

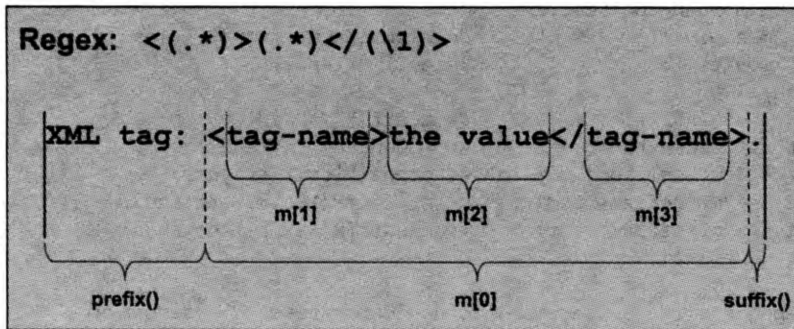


Рис. 14.1. Интерфейс соответствия регулярного выражения

После вызова объект `m` типа `match_results` имеет состояние, которое показано на рис. 14.1 и обеспечивает следующий интерфейс.

- Объект класса `match_results` содержит:
 - объект `m[0]` типа `sub_match` для всех соответствующих символов;
 - объект `prefix()` типа `sub_match`, представляющий все символы, предшествующие первому соответствующему символу;
 - объект `suffix()` типа `sub_match`, представляющий все символы, следующие после последнего соответствующего символа.
- Кроме того, любая “группа захвата” имеет доступ к соответствующему объекту `m[i]` типа `sub_match`. Поскольку в данном примере регулярное выражение определяет три “группы захвата” (одна для начального дескриптора, одна для значения и одна для заключительного дескриптора), они доступны как `m[1]`, `m[2]` и `m[3]`.
- Функция `size()` возвращает количество объектов типа `sub_match` (включая `m[0]`).
- Все объекты типа `sub_match` порождены из класса `pair<>` и содержат первый символ как член `first`, и позицию, следующую за последним символом, как член `second`. Кроме того, функция `str()` возвращает символы как строку, функция `length()` возвращает количество символов, оператор `<<` записывает символы в поток, а кроме того, для них определено неявное преобразование в строку.
- Кроме того, объект класса `match_results` содержит:

- функцию-член `str()` для создания строки в целом (с помощью вызова `str()` или `str(0)`) или n -й соответствующей подстроки (с помощью вызова `str(n)`), которая пуста, если соответствующих подстрок не существует (таким образом, допускается передача параметра n , который превышает `size()`);
- функцию-член `length()` для вычисления длины соответствующей строки в целом (с помощью вызова `length()` или `length(0)`) или длины n -й соответствующей подстроки (с помощью вызова `length(n)`), которая равна 0, если соответствующих подстрок не существует (таким образом, допускается передача параметра n , который превышает `size()`);
- функцию-член `position()` для вычисления позиции соответствующей подстроки в целом (с помощью вызова `position()` или `position(0)`) или позиции n -й соответствующей подстроки (с помощью вызова `length(n)`);
- функции-члены `begin()`, `cbegin()`, `end()` и `cend()` для перебора объектов от `m[0]` до `m[n]` типа `sub_match`;

Программа выводит следующий результат:

```
data:                XML tag: <tag-name>the value</tag-name>.

m.empty():           false
m.size():             4
m.str():              <tag-name>the value</tag-name>
m.length():          30
m.position():         9
m.prefix().str():    XML tag:
m.suffix().str():    .
m[0].str():           <tag-name>the value</tag-name>
m.str(0):             <tag-name>the value</tag-name>
m.position(0):        9
m[1].str():           tag-name
m.str(1):             tag-name
m.position(1):        10
m[2].str():           the value
m.str(2):             the value
m.position(2):        19
m[3].str():           tag-name
m.str(3):             tag-name
m.position(3):        30

matches:
<tag-name>the value</tag-name> (length: 30)
tag-name (length: 8)
the value (length: 9)
tag-name (length: 8)
```

Иначе говоря, существуют четыре способа вычисления соответствующей строки в целом в объекте `match_result<> m`:

```
m.str()              // вычисляет соответствующую строку в целом
m.str(0)              // то же самое
m[0].str()            // то же самое
*(m.begin())          // то же самое
```

и три способа вычисления n -й соответствующей подстроки, если она есть:

```
m.str(1)           // вычисляет первую соответствующую подстроку, если она есть,
                   // или "" в противном случае
m[1].str()          // то же самое
*(m.begin()+1)      // вычисляет первую соответствующую подстроку, если она есть,
                   // в противном случае результат некорректный
```

Если вызвать `regex_match()` вместо `regex_search()`, то интерфейс объекта типа `match_results` будет таким же. Однако, поскольку функция `regex_match()` всегда сравнивает всю символьную строку, префикс и суффикс всегда будут пустыми.

Теперь у нас есть вся информация, необходимая для *всех* соответствий регулярного выражения, как показано в следующей программе:

```
// regex/regex3.cpp

#include <string>
#include <regex>
#include <iostream>
using namespace std;

int main()
{
    string data = "<person>\n"
                  " <first>Nico</first>\n"
                  " <last>Josuttis</last>\n"
                  "</person>\n";

    regex reg("<(.*?)>(.*?)</(\\1)>");

    // перебор всех соответствий
    auto pos=data.cbegin();
    auto end=data.cend();
    smatch m;
    for ( ; regex_search(pos,end,m,reg); pos=m.suffix().first) {
        cout << "match: " << m.str() << endl;
        cout << " tag:   " << m.str(1) << endl;
        cout << " value: " << m.str(2) << endl;
    }
}
```

Здесь мы используем регулярное выражение (дополнительная обратная косая черта в строчном литерале C++ игнорируется) `<(.*?)>(.*?)</(\\1)>` для поиска строки

```
<anyNumberOfAnyChars1>anyNumberOfAnyChars2</anyNumberOfAnyChars1>
```

Таким образом, мы ищем дескрипторы XML (`\\1` значит: *то же, что и первая соответствующая подстрока*).

В нашем примере данное регулярное выражение используется с помощью другого интерфейса, выполняющего перебор соответствующих символьных строк. По этой причине вместо передачи всей символьной строки мы передаем диапазон соответствующих элементов. Мы начинаем с диапазона, содержащего все символы, используя функции `cbegin()` и `cend()` в строке поиска:

```
auto pos=data.cbegin();
auto end=data.cend();
```

Затем, после каждого сопоставления, мы продолжаем поиск с начала оставшихся символов:

```
smatch m;
for ( ; regex_search(pos,end,m,reg); pos=m.suffix().first) {
    ...
}
```

Таким образом, поскольку анализируемая строка `data` имеет значение

```
<person>
<first>Nico</first>
<last>Josuttis</last>
</person>
```

программа выводит следующий результат:

```
match: <first>Nico</first>
tag: first
value: Nico
match: <last>Josuttis</last>
tag: last
value: Josuttis
```

Для повторной инициализации итератора `pos` можно было бы передать аргумент `m[0].second()` (конец сопоставляемых символов), а не выражение `m.suffix().first`. В обоих случаях следует использовать итераторы `const_iterator`. Таким образом, компилятор не допустит использование функции `begin()` и `end()` для инициализации итераторов `pos` и `end`.

Вывод может получиться другим, если дескрипторы в строке `data` не разделены символом перехода на новую строку:

```
<person><first>Nico</first><last>Josuttis</last></person>
```

Итак, вывод может получиться таким:

```
match: <person><first>Nico</first><last>Josuttis</last></person>
tag: person
value: <first>Nico</first><last>Josuttis</last>
```

Причина заключается в том, что функции для работы с регулярными выражениями являются *жадными*. Иначе говоря, возвращается *самое длинное соответствие*. При использовании символов перехода на новую строку дескриптор, открывающийся символами `<person>`, может не совпасть с искомым, потому что мы ищем “.” в качестве значения, т.е. “любое количество любых символом, за исключением перехода на новую строку”. Без использования символов перехода на новую строку весь дескриптор с символами `<person>` соответствует шаблону. Чтобы гарантировать, что мы сможем по-прежнему находить внутренние дескрипторы, мы должны были бы изменить регулярное выражение, например:

```
"<(.*>([>]*)</(\1)>"
```

В качестве значения мы теперь ищем группу “[^>]*”, т.е. “любое количество любых символов, кроме >”. Следовательно, внутренние дескрипторы больше не считаются частью значения.

14.3. Итераторы регулярных выражений

Для того чтобы перебрать все соответствия при регулярном поиске, можно использовать итераторы регулярных выражений. Эти итераторы имеют тип `regex_iterator<>` и обычные конкретизации для строки и символьных последовательностей с префиксами `s`, `c`, `ws` или `wc`. Рассмотрим следующий пример:

```
// regex/regexiter1.cpp

#include <string>
#include <regex>
#include <iostream>
#include <algorithm>
using namespace std;

int main()
{
    string data = "<person>\n"
                  "<first>Nico</first>\n"
                  "<last>Josuttis</last>\n"
                  "</person>\n";

    regex reg("<(.*?)>(.*?)</(\\1)>");

    // перебор всех соответствий (с помощью итератора regex_iterator):
    sregex_iterator pos(data.cbegin(), data.cend(), reg);
    sregex_iterator end;
    for ( ; pos != end ; ++pos ) {
        cout << "match: " << pos->str() << endl;
        cout << " tag: " << pos->str(1) << endl;
        cout << " value: " << pos->str(2) << endl;
    }

    // использование итератора regex_iterator для обработки каждой
    // соответствующей подстроки как элемента в алгоритме
    sregex_iterator beg(data.cbegin(), data.cend(), reg);
    for_each (beg, end, [] (const smatch& m) {
        cout << "match: " << m.str() << endl;
        cout << " tag: " << m.str(1) << endl;
        cout << " value: " << m.str(2) << endl;
    });
}
```

Здесь выражение

```
sregex_iterator pos (data.cbegin(), data.cend(), reg);
```

инициализирует итератор регулярных выражений, выполняющий обход данных для поиска соответствий `reg`. Конструктор по умолчанию для этого типа определяет итератор, установленный на позицию, следующую за последней:

```
sregex_iterator end;
```

Теперь этот итератор можно использовать как любой другой двунаправленный итератор (см. раздел 9.2.4): операция `*` возвращает текущее соответствие, а операции `++` и `--` перемещают итератор на следующее или предыдущее соответствие. Таким образом, следующие операторы выводят все соответствия, их дескрипторы и значения (как в предыдущем примере):

```
for ( ; pos!=end ; ++pos ) {
    cout << "match: " << pos->str() << endl;
    cout << " tag:   " << pos->str(1) << endl;
    cout << " value: " << pos->str(2) << endl;
}
```

Разумеется, такой итератор можно использовать и в алгоритме. Таким образом, следующий фрагмент программы выполняет вызов лямбда-функции, передаваемой как третий аргумент, при каждом соответствии (подробное описание лямбда-функций и алгоритмов см. в разделе 6.9):

```
// использование итератора regex_iterator для обработки каждой
// соответствующей подстроки как элемента в алгоритме
sregex_iterator beg(data.cbegin(), data.cend(), reg);
sregex_iterator end;
for_each (beg, end, [](const smatch& m) {
    cout << "match: " << m.str() << endl;
    cout << " tag:   " << m.str(1) << endl;
    cout << " value: " << m.str(2) << endl;
});
```

14.4. Итераторы токенов регулярных выражений

Итератор регулярных выражений позволяет перебирать все соответствующие подстроки. Однако иногда необходимо обрабатывать все содержимое между соответствующими выражениями. Например, такая ситуация часто возникает, когда требуется разбить строки на отдельные токены, разделенные чем-нибудь, возможно, даже заданным как регулярное выражение. Эту функциональную возможность предоставляет класс `regex_token_iterator<>`, имеющий обычные конкретизации для строки символьных последовательностей с префиксами `s`, `c`, `ws` или `wc`.

И вновь, для инициализации мы можем передать начало и конец символьной последовательности и регулярное выражение. Кроме того, можно задать список целочисленных значений, представляющих элементы “токенизации”.

- `-1` означает, что нас интересуют все подпоследовательности между соответствующими регулярными выражениями (разделители токенов).

- 0 означает, что нас интересуют все соответствующие регулярные выражения (разделители токенов).
- Все другие значения n означают, что нас интересует n -е соответствующее подвыражение в регулярных выражениях.

Рассмотрим следующий пример:

```
// regex/regex_tokeniter1.cpp

#include <string>
#include <regex>
#include <iostream>
#include <algorithm>
using namespace std;

int main()
{
    string data = "<person>\n"
                  " <first>Nico</first>\n"
                  " <last>Josuttis</last>\n"
                  "</person>\n";

    regex reg("<(.*?)>(.*?)</(\\1)>");

    // перебираем все соответствия (с помощью итератора regex_token_iterator)
    sregex_token_iterator pos(data.cbegin(), data.cend(), // последовательность
                             reg,                        // разделитель токенов
                             {0, 2});                   // 0: полное соответствие,
                                                         // 2: вторая подстрока

    sregex_token_iterator end;
    for ( ; pos != end ; ++pos ) {
        cout << "match: " << pos->str() << endl;
    }
    cout << endl;

    string names = "nico, jim, helmut, paul, tim, john paul, rita";
    regex sep("[ \\t\\n]*[,;\\.][ \\t\\n]*"); // разделенные , ; или . и пропусками
    sregex_token_iterator p(names.cbegin(), names.cend(), // последовательность
                           sep,                          // разделитель
                           -1);                          // -1: значения между разделителями

    sregex_token_iterator e;
    for ( ; p != e ; ++p ) {
        cout << "name: " << *p << endl;
    }
}
```

Программа выводит следующий результат:

```
match: <first>Nico</first>
match: Nico
match: <last>Josuttis</last>
match: Josuttis
```



```

name: nico
name: jim
name: helmut
name: paul
name: tim
name: john paul
name: rita

```

Здесь итератор токенов регулярных выражений для объектов класса `string` (префикс `s`) инициализируется символьной последовательностью `data`, регулярным выражением `reg` и списком из двух индексов (0 и 2):

```

sregex_token_iterator pos(data.cbegin(), data.cend(), // последовательность

                        reg,                          // разделитель токенов

                        {0, 2});                      // 0: полное соответствие,
                                                    // 2: вторая подстрока

```

Список индексов указывает, что нас интересуют все соответствия и вторые подстроки каждого соответствия.

Обычное применение такого итератора токенов регулярных выражений демонстрирует следующая итерация. В данном случае используется список имен:

```
string names = "nico, jim, helmut, paul, tim, john paul, rita";
```

Теперь регулярное выражение определяет разделители этих имен. В данном случае это запятая, или точка с запятой, или точка с необязательными пропусками (пробелами, символами табуляции и символами перехода на новую строку):

```
regex sep("[ \\t\\n]*[,;.] [\\t\\n]*"); // разделенные , ; или . и пропусками
```

В качестве альтернативы можно использовать следующее регулярное выражение (см. раздел 14.8):

```
regex sep("[[:space:]]*[,;.] [[:space:]]*"); // разделенные , ; или .
                                           // и пропусками
```

Поскольку нас интересуют только значения между разделителями токенов, программа выводит все имена из списка (удаляя пробелы).

Отметим, что интерфейс класса `sregex_token_iterator` позволяет задавать искомые токены разными способами.

- Можно передать отдельное целочисленное значение.
- Можно передать список инициализации или целочисленные значения (см. раздел 3.1.3).
- Можно передать объект класса `vector`, состоящий из целочисленных значений.
- Можно передать массив целочисленных значений.

14.5. Замена регулярных выражений

В заключение рассмотрим интерфейс, позволяющий заменять символьные последовательности, соответствующие регулярному выражению. Рассмотрим следующий пример:

```
// regex/regexreplace1.cpp

#include <string>
#include <regex>
#include <iostream>
#include <iterator>
using namespace std;

int main()
{
    string data = "<person>\n"
                 "  <first>Nico</first>\n"
                 "  <last>Josuttis</last>\n"
                 "</person>\n";
    regex reg("<(.*?)>(.*?)</(\\1)>");

    // выводим данные с заменой соответствий шаблонам
    cout << regex_replace (data,                // данные
                           reg,                  // регулярное выражение
                           "<$1 value=\"$2\"/>")    // замена
          << endl;

    // то же самое с помощью синтаксиса sed
    cout << regex_replace (data,                // данные
                           reg,                  // регулярное выражение
                           "<\\1 value=\"$2\"/>",    // замена
                           regex_constants::format_sed) // флаг формата
          << endl;

    // используем интерфейс итераторов и // - format_no_copy: не копируем
    символы, которые не соответствуют шаблону
    // - format_first_only: заменяем только первое найденное соответствие
    string res2;
    regex_replace (back_inserter(res2),          // получатель
                  data.begin(), data.end(),      // диапазон источника
                  reg,                            // регулярное выражение
                  "<$1 value=\"$2\"/>",          // замена
                  regex_constants::format_no_copy | // флаги формата
                  regex_constants::format_first_only);
    cout << res2 << endl;
}
```

Здесь мы снова используем регулярное выражение для сравнения значений с XML/HTML-дескрипторами. Однако на этот раз мы преобразовываем ввод в следующий вывод:

```
<person>
  <first value="Nico"/>
  <last value="Josuttis"/>
```

```
</person>

<person>
  <first value="Nico"/>
  <last value="Josuttis"/>
</person>

<first value="Nico"/>
```

Для этого задаем замену, в которой можно использовать соответствующие шаблонам подвыражения с символом \$ (см. табл. 14.1). Здесь \$1 и \$2 используют дескриптор и значение, найденное в заменяющей строке:

```
"<$1 value="\$2"/>" // замена с помощью синтаксиса по умолчанию
```

Неформатированная строка позволяет избежать использования кавычек:

```
R"(<$1 value="$2"/>)" // замена с помощью синтаксиса по умолчанию
```

Передавая константу `regex_constants::format_sed`, можно воспользоваться синтаксисом замены команды UNIX `sed` (см. второй столбец в табл. 14.1).

```
"<\\1 value=\\"\\2\\"/>" // замена с помощью синтаксиса sed
```

И снова с помощью неформатированной строки мы можем избежать использования обратных косых черт:

```
R"(<\\1 value="\2"/>)" // замена с помощью синтаксиса sed,
                        // заданная как неформатированная строка
```

Таблица 14.1. Символы замены в регулярных выражениях

Шаблон по умолчанию		Смысл шаблона sed
\$&	&	Шаблон сравнения
\$n	\n	n-я группа захвата для сравнения
\$'		Префикс шаблона сравнения
\$'		Суффикс шаблона сравнения
\$\$		Символ \$

14.6. Флаги регулярных выражений

Мы уже ввели некоторые константы регулярных выражений, с помощью которых можно влиять на работу интерфейса регулярных выражений.

```
regex reg3("<\\(.*)>.*</\\1>", regex_constants::grep); // используем
                                                         // грамматику grep

regex_replace (data, reg,
               string("<\\1 value=\\"\\2\\"/>"),
               regex_constants::format_sed) // используем синтаксис замены sed
```

Однако этих констант много больше. В табл. 14.2 перечислены все константы регулярных выражений, содержащиеся в библиотеке `regex`, а также указано, где их можно использовать. В принципе их всегда можно передать конструктору или функции регулярного выражения как необязательный последний аргумент.

Таблица 14.2. Константы регулярных выражений в пространстве имен `std::regex_constants`

Константы <code>regex_constant</code>	Смысл
Грамматика регулярных выражений	
<code>ECMAScript</code>	Используется грамматика ECMAScript (по умолчанию)
<code>Basic</code>	Используется грамматика базовых регулярных выражений (BRE) системы POSIX
<code>Extended</code>	Используется грамматика расширенных регулярных выражений (ERE) системы POSIX
<code>Awk</code>	Используется грамматика инструмента UNIX <code>awk</code>
<code>Grep</code>	Используется грамматика инструмента UNIX <code>grep</code>
<code>Egrep</code>	Используется грамматика инструмента UNIX <code>egrep</code>
Другие флаги создания	
<code>icase</code>	Нечувствительность к регистру
<code>nosubs</code>	Не хранить подпоследовательности в результатах соответствий
<code>optimize</code>	Оптимизировать скорость сравнения, а не скорость создания регулярного выражения
<code>collate</code>	Диапазоны символов в виде <code>[a-b]</code> будут зависеть от локального контекста
Флаги алгоритма	
<code>match_not_null</code>	Пустая последовательность не будет соответствием
<code>match_not_bol</code>	Первый символ не будет соответствовать <i>началу строки</i> (шаблон <code>^</code>)
<code>match_not_eol</code>	Последний символ не будет соответствовать <i>концу строки</i> (шаблон <code>\$</code>)
<code>match_not_bow</code>	Первый символ не будет соответствовать <i>началу слова</i> (шаблон <code>\b</code>)
<code>match_not_eow</code>	Последний символ не будет соответствовать <i>концу слова</i> (шаблон <code>\b</code>)
<code>match_continuous</code>	Выражение будет соответствовать только подпоследовательности, которая начинается с первого символа
<code>match_any</code>	Если есть несколько соответствий, то приемлемым считается любое
<code>match_prev_avail</code>	Позиции перед первым символом считаются корректными (игнорируют константы <code>match_not_bol</code> и <code>match_not_bow</code>)
Флаги замены	
<code>format_default</code>	Использовать синтаксис замены по умолчанию (ECMAScript)
<code>format_sed</code>	Использовать синтаксис замены инструмента UNIX <code>sed</code>
<code>format_first_only</code>	Заменять только первое соответствие
<code>format_no_copy</code>	Не копировать символы, не соответствующие шаблону

Рассмотрим небольшую программу, демонстрирующую применение некоторых флагов:

```
// regex/regex4.cpp

#include <string>
#include <regex>
#include <iostream>
using namespace std;

int main()
{
    // поиск записей в предметном указателе LaTeX без учета регистра
    string pat1 = R"(\\.index\[([^\]]*)\])"; // первая группа захвата
    string pat2 = R"(\\.index\[([^\]]*)\]([^\]]*)\])"; // 2-я и 3-я группы захвата
    regex pat (pat1+"\n"+pat2,
               regex_constants::egrep|regex_constants::icase);

    // инициализируем строку символами из стандартного ввода
    string data((istreambuf_iterator<char>(cin),
        istreambuf_iterator<char>()));

    // ищем и выводим соответствующие записи в предметном указателе
    smatch m;
    auto pos = data.cbegin();
    auto end = data.cend();
    for ( ; regex_search (pos,end,m,pat); pos=m.suffix().first) {
        cout << "match: " << m.str() << endl;
        cout << " val:  " << m.str(1)+m.str(2) << endl;
        cout << " see:  " << m.str(3) << endl;
    }
}
```

Цель программы — найти записи в предметном указателе LATEX, которые могут иметь один или два аргумента. Кроме того, записи могут быть набраны в нижнем или верхнем регистре. Итак, требуется найти запись, соответствующую следующим условиям.

- Обратная косая черта, за которой следуют какие-то символы и слово `index` (в нижнем или верхнем регистре), а также запись предметного указателя, заключенная в фигурные скобки, как показано ниже.

```
\index{STL}%
\MAININDEX{standard template library}%
```

- Обратная косая черта, за которой следуют какие-то символы и слово `index` (в верхнем или нижнем регистре), а также запись предметного указателя и слова “see also”, заключенные в фигурные скобки, как показано ниже.

```
\SEEINDEX{standard template library}{STL}%
```

Используя грамматику `egrep`, между двумя регулярными выражениями можно поместить символ перехода на новую строку. (Фактически команды `grep` и `egrep` могут одновременно искать несколько регулярных выражений, заданных в разных строках.) Однако в этом случае следует учитывать свойство *жадности*, т.е. необходимо гарантировать, что первое регулярное выражение не соответствует последовательностям, которые соответствуют второму регулярному выражению. Итак, вместо того, чтобы допускать любые

символы в записи предметного указателя, мы должны гарантировать, что фигурных скобок не будет. В результате получаются следующие регулярные выражения:

```
\\.*index\\{([\\^])*}\\}
\\.*index\\{(.*)\\}\\{(.*)\\}
```

Их можно также представить в виде неформатированных строк

```
R"\\.*index\\{([\\^])*}\\}"
R"\\.*index\\{(.*)\\}\\{(.*)\\}"
```

или обычных строковых литералов

```
"\\\\\\\\.*index\\\\{([\\^])*\\\\}"
"\\\\\\\\.*index\\\\{(.*)\\\\}\\{(.*)\\\\}"
```

Окончательное регулярное выражение мы создаем, выполняя конкатенацию обоих выражений и передавая флаги для использования грамматики, в которой символ `\n` разделяет альтернативные шаблоны (см. раздел 14.9) и игнорируется регистр.

```
regex pat (pat1+"\n"+pat2,
           regex_constants::egrep|regex_constants::icase);
```

В качестве входной информации мы используем все символы, считанные из стандартного потока ввода. Здесь мы используем строку `data`, которая инициализируется началом и концом всех считанных символов (см. разделы 7.1.2 и 15.13.2).

```
string data((istreambuf_iterator<char>(cin)),
            istreambuf_iterator<char>());
```

Отметим, что первое регулярное выражение имеет одну “группу захвата”, а второе — две. Таким образом, если первое регулярное выражение соответствует записи предметного указателя, то эта запись оказывается в первой подгруппе. Если записи предметного указателя соответствует второе регулярное выражение, то эта запись окажется во втором частичном соответствии, а слова “see also” — в третьем. По этой причине мы выводим в качестве найденного значения содержимое первого частичного соответствия, а затем второго (одно частичное соответствие будет содержать значение, а второе будет пустым).

```
smatch m;
auto pos = data.begin();
auto end = data.end();
for ( ; regex_search (pos,end,m,pat); pos=m.suffix().first) {
    cout << "match: " << m.str() << endl;
    cout << " val:  " << m.str(1)+m.str(2) << endl;
    cout << " see:  " << m.str(3) << endl;
}
```

Вызовы `str(2)` и `str(3)` являются корректными, даже если соответствий не существует. Функция `str()` в этом случае обязательно выдаст пустую строку.

При вводе строк

```
\chapter{The Standard Template Library}
\index{STL}%
\MAININDEX{standard template library}%
\SEEINDEX{standard template library}{STL}%
This is the basic chapter about the STL.
\section{STL Components}
```

```
\hauptindex{STL, introduction}%
The \stl{} is based on the cooperation of
...
```

программа выводит на экран следующий результат:

```
match: \index{STL}
  val: STL
  see:
match: \MAININDEX{standard template library}
  val: standard template library
  see:
match: \SEEINDEX{standard template library}{STL}
  val: standard template library
  see: STL
match: \hauptindex{STL, introduction}
  val: STL, introduction
  see:
```

14.7. Исключения, связанные с регулярными выражениями

При разборе регулярных выражений ситуация может стать очень сложной. Стандартная библиотека C++ содержит специальный класс исключений для работы с регулярными выражениями. Этот класс является производным от класса `std::runtime_error` (см. раздел 4.3.1) и содержит дополнительную функцию-член `code()` для возвращения кода ошибки. Это может помочь при поиске ошибки в случае возникновения исключения при разборе регулярных выражений.

К сожалению, коды ошибок, возвращаемые функцией `code()`, зависят от реализации, поэтому выводить их на экран не имеет смысла. Вместо этого необходимо использовать следующий заголовочный файл (или подобный ему) для разумной обработки исключений, связанных с регулярными выражениями:

```
// regex/regexexception.hpp

#include <regex>
#include <string>

template <typename T>
std::string regexCode (T code)
{
    switch (code) {
        case std::regex_constants::error_collate:
            return "error_collate: "
                "regex has invalid collating element name";
        case std::regex_constants::error_ctype:
            return "error_ctype: "
                "regex has invalid character class name";
        case std::regex_constants::error_escape:
            return "error_escape: "
                "regex has invalid escaped char. or trailing escape";
        case std::regex_constants::error_backref:
```

```

        return "error_backref: "
            "regex has invalid back reference";
    case std::regex_constants::error_brack:
        return "error_brack: "
            "regex has mismatched '[' and ']'";
    case std::regex_constants::error_paren:
        return "error_paren: "
            "regex has mismatched '(' and ')'";
    case std::regex_constants::error_brace:
        return "error_brace: "
            "regex has mismatched '{' and '}'";
    case std::regex_constants::error_badbrace:
        return "error_badbrace: "
            "regex has invalid range in {} expression";
    case std::regex_constants::error_range:
        return "error_range: "
            "regex has invalid character range, such as '[b-a]'";
    case std::regex_constants::error_space:
        return "error_space: "
            "insufficient memory to convert regex into finite state";
    case std::regex_constants::error_badrepeat:
        return "error_badrepeat: "
            "one of *?+{ not preceded by valid regex";
    case std::regex_constants::error_complexity:
        return "error_complexity: "
            "complexity of match against regex over pre-set level";
    case std::regex_constants::error_stack:
        return "error_stack: "
            "insufficient memory to determine regex match";
    }
    return "unknown/non-standard regex error code";
}

```

Подробное объяснение, записанное в скобках после имени кода ошибки, взято непосредственно из спецификации стандартной библиотеки C++. Использование этого заголовочного файла демонстрируется в следующей программе:

```

// regex/regex5.cpp

#include <regex>
#include <iostream>
#include "regexexception.hpp"
using namespace std;

int main()
{
    try {
        // инициализируем регулярное выражение
        // некорректной синтаксической конструкцией
        regex pat ("\\\\.*index\\\\{([\\^}]*)\\\\}",
            regex_constants::grep|regex_constants::icase);
        ...
    }
    catch (const regex_error& e) {
        cerr << "regex_error: \\n"
            << " what(): " << e.what() << "\\n"

```



```

    << " code(): " << regexCode(e.code()) << endl;
  }
}

```

Поскольку мы используем грамматику `grep`, но прибегаем к управляющим символам перед фигурными скобками, `{ }`, программа может вывести следующий результат:

```

regex_error:
what(): regular expression error
code(): error_badbrace: regex has invalid range in {} expression

```

14.8. Грамматика ECMAScript

Грамматикой по умолчанию в библиотеке регулярных выражений является модифицированная грамматика ECMAScript (см. [ECMAScript]) — самая мощная среди всех грамматик. В табл. 14.3 перечислены наиболее важные из специальных выражений и указан их смысл.

Таблица 14.3. Общие регулярные выражения для грамматики по умолчанию (ECMAScript)

Выражение	Смысл
.	Любой символ, кроме символа перехода на новую строку
[...]	Один из символов ... (может содержать диапазоны)
[^...]	Ни один из символов ... (может содержать диапазоны)
[[: <i>charclass</i> :]]	Символ указанного символьного класса <i>charclass</i> (см. табл. 14.4)
\n, \t, \f, \r, \v	Символ перехода на новую строку, табуляции, прогона страницы, перевода каретки, возврата или вертикальной табуляции
\xhh, \uhhh	Шестнадцатеричный символ или символ системы Unicode
\d, \D, \s, \S, \w, \W	Сокращение для символа символьного класса (см. табл. 14.4)
*	Предыдущий символ или группа произвольное количество раз
?	Предыдущий символ или группа — необязательно (ни разу или один раз)
+	Предыдущий символ или группа хотя бы один раз
{n}	Предыдущий символ или группа <i>n</i> раз
{n, }	Предыдущий символ или группа хотя бы <i>n</i> раз
{n, m}	Предыдущий символ или группа не менее <i>n</i> и не более <i>m</i> раз
... ...	Шаблон до символа или шаблон после него
(...)	Группировка
\1, \2, \3, ...	<i>n</i> -я группа (первая группа имеет индекс 1)
\b	Положительная граница слова (начало или конец слова)
\B	Отрицательная граница слова (не начало и не конец слова)
^	Начало строки (включая начало всех символов)
\$	Конец строки (включая конец всех символов)

В квадратных скобках можно указывать любую комбинацию символов (включая специальные символы), диапазоны символов (например, [0-9a-z]) и символьные классы (например, [[:digit:]]). Начальный символ ^ отрицает все выражение, так что все выражение в квадратных скобках означает “любой символ, за исключением ...”. В табл. 14.4 приведены все возможные символьные классы для регулярных выражений. Отметим, что базовые классы соответствуют вспомогательным функциям для символьных классификаций в разделе 16.4.4. Однако однобуквенные сокращения поддерживаются только регулярными выражениями. Управляющие последовательности классов символов поддерживаются только грамматикой ECMAScript.

Ниже приведено несколько примеров

```
[[:alpha:]][_[:alnum:]]*           // идентификатор C++
(?:\n)*                          // любое количество любых символов
                                  // (включая переход на новую строку)
[123]?[0-9]\.1?[0-9]\.20[0-9]{2} // дата первого века второго тысячелетия
                                  // (немецкий формат, например, 24.12.2010)
```

Таблица 14.4. Символьные классы и соответствующие управляющие последовательности (ECMAScript)

Символ	Сокращенное название класса	Управляющая последовательность	Описание
[[:alnum:]]			Буква или цифра (эквивалент [[:alpha:][:digit:]])
[[:alpha:]]			Буква
[[:blank:]]			Пробел или табуляция
[[:cntrl:]]			Управляющий символ
[[:digit:]]	[[:d:]]	\d	Цифра
		\D	Не цифра (эквивалент [^[:digit:]])
[[:graph:]]			Печатаемый непробельный символ (эквивалент [[:alnum:][:punct:]])
[[:lower:]]			Буква в нижнем регистре
[[:print:]]			Печатаемый символ (включая пропуски)
[[:punct:]]			Знаки пунктуации (т.е. печатаемые символы, но не пробел, цифра или буква)
[[:space:]]	[[:s:]]	\s	Пробел
		\S	Не пробел (эквивалент [^[:space:]])
		[[:upper:]]	Буква в верхнем регистре
[[:xdigit:]]			Шестнадцатеричная буква
	[[:w:]]	\w	Буква, цифра или символ подчеркивания (эквивалент [[:alpha:][:digit:]_])
		\W	Не буква, не цифра и не символ подчеркивания (эквивалент [^[:alpha:][:digit:]_])

14.9. Другие грамматики

Помимо грамматики ECMAScript, стандартная библиотека C++ поддерживает пять других грамматик, которые можно задать с помощью соответствующих констант регулярных выражений (см. раздел 14.6).

- ECMAScript: грамматика ECMAScript, установленная по умолчанию;
- basic: грамматика базовых регулярных выражений (BRE) POSIX;
- extended: грамматика расширенных регулярных выражений (ERE) POSIX;
- awk: грамматика инструмента UNIX awk;
- grep: грамматика инструмента UNIX grep;
- egrep: грамматика инструмента UNIX egrep.

Основные различия между этими грамматиками приведены в табл. 14.5. Как видим, грамматика ECMAScript намного мощнее остальных. Немногочисленные возможности, которые она не поддерживает, — это использование символа перехода на новую строку для разделения нескольких шаблонов с помощью операции “или,” как это принято в грамматиках grep и egrep, и возможность инструмента awk задавать восьмеричные управляющие последовательности.

Таблица 14.5. Различия между грамматиками регулярных выражений

Возможность	ECMA-Script	basic	extended	awk	grep	egrep
Символы для группировки	()	\ (\)	()	()	\ (\)	()
Символы для повторений	{ }	\ { \}	{ }	{ }	\ { \}	{ }
? значит “ни разу или один раз”	Да	—	Да	Да	—	Да
+ значит “хотя бы один раз”	Да	—	Да	Да	—	Да
значит “или”	Да	—	Да	Да	—	Да
\n разделяет альтернативные шаблоны	—	—	—	—	Да	Да
\n ссылается на группу n	Да	Да	—	—	Да	—
Границы слова (\b and \B)	Да	—	—	—	—	—
Шестнадцатеричный или Unicode управляющие последовательности	Да	—	—	—	—	—
Управляющие последовательности символьных классов	Да	—	—	—	—	—
\n, \t, \f, \r, \v	Да	—	—	Да	—	—
\a (сигнал) или \b (возврат)	—	—	—	Да	—	—
\ooo для восьмеричных значений	—	—	—	Да	—	—

14.10. Подробное описание основных сигнатур регулярных выражений

В табл. 14.6 приведены сигнатуры основных операций над регулярными выражениями `regex_match()` (см. раздел 14.1), `regex_search()` (см. раздел 14.1) и `regex_replace()` (см. раздел 14.5). Легко видеть, что существуют перегрузки для операций над строками, которые могут быть как объектами класса `basic_string<>`, так и обычными C-строками, например строковыми литералами, а также для итераторов, задающих начало и конец обрабатываемой последовательности. Кроме того, всегда можно задать формат в качестве необязательного последнего аргумента.

Если соответствие найдено, то функции `regex_match()` и `regex_search()` возвращают значение `true`. Кроме того, они позволяют передавать необязательный аргумент `matchRet`, содержащий детальную информацию о найденных соответствиях. Эти аргументы типа `std::match_results<>` (см. раздел 14.2) должны конкретизироваться типом итератора, соответствующим символьному типу.

- Для строк C++ он соответствует константному итератору. Для типов `string` и `wstring` соответствующие типы `smatch` и `wsmatch` определяются следующим образом:

```
typedef match_results<string::const_iterator> smatch;
typedef match_results<wstring::const_iterator> wsmatch;
```

- Для C-строк, включая строковые литералы, он соответствует типу указателя. Для C-строк, состоящих из символов типа `char` или `wchar_t`, определены соответствующие типы `cmatch` и `wcmatch`.

```
typedef match_results<const char*> cmatch;
typedef match_results<const wchar_t*> wcmatch;
```

Для функции `regex_replace()` необходимо передать спецификацию *repl* в виде строки (для нее также существуют перегруженные варианты как для класса `basic_string<>`, так и для обычных C-строк, например строковых литералов). Версия для строки возвращает новую строку с соответствующими заменами. Версия для итератора возвращает первый аргумент *outPos*, который должен быть выходным итератором, указывающим место для записи с замещением.

Таблица 14.6. Сигнатуры операций над регулярными выражениями

Сигнатура	Описание
<code>bool regex_match(str, regex)</code>	Проверяет полное соответствие <i>regex</i>
<code>bool regex_match(str, regex, flags)</code>	
<code>bool regex_match(beg, end, regex)</code>	
<code>bool regex_match(beg, end, regex, flags)</code>	
<code>bool regex_match(str, matchRet, regex)</code>	

Окончание табл. 14,6

Проверяет и возвращает полное соответствие *regex*

<code>bool regex_match(str, matchRet, regex, flags)</code>	Проверяет и возвращает полное соответствие <i>regex</i>
<code>bool regex_match(beg, end, matchRet, regex)</code>	
<code>bool regex_match(beg, end, matchRet, regex, flags)</code>	

<code>bool regex_search(str, regex)</code>	Ищет соответствие <i>regex</i>
<code>bool regex_search(str, regex, flags)</code>	
<code>bool regex_search(beg, end, regex)</code>	
<code>bool regex_search(beg, end, regex, flags)</code>	
<code>bool regex_search(str, matchRet, regex)</code>	

<code>bool regex_search(str, matchRet, regex, flags)</code>	Ищет и возвращает соответствие <i>regex</i>
<code>bool regex_search(beg, end, matchRet, regex)</code>	
<code>bool regex_search(beg, end, matchRet, regex, flags)</code>	
<code>strRes regex_replace(str, regex, repl)</code>	

<code>strRes regex_replace(str, regex, repl, flags)</code>	Заменяет соответствия согласно <i>regex</i>
<code>outPos regex_replace(outPos, beg, end, regex, repl)</code>	
<code>outPos regex_replace(outPos, beg, end, regex, repl, flags)</code>	

В заключение заметим, что, для того чтобы избежать недоразумений, стандарт не предусматривает неявное преобразование строк и строковых литералов в тип `regex`. Таким образом, программист всегда должен самостоятельно явно преобразовывать строки, содержащие регулярное выражение, в тип `std::regex` (или `std::basic_regex<>`).

